

# Railway Transport & Reservation Management System

Database Project — Phase 2 Submission

Group: 202401068 | DAIICT

## **Submission Contents:**

- (1) Normalized Relational Schema Diagram
- (2) Minimal Functional Dependency Set
- (3) Normalization Proofs (BCNF)
- (4) DDL Scripts for PostgreSQL

## Part 1: Normalized Relational Schema

The following relational schema is derived from the ER diagram by applying the standard ER-to-Relational mapping rules. Each entity becomes a relation, each many-to-many relationship becomes a relation, ISA hierarchies are mapped using the shared primary key approach, and weak entities carry the PK of their identifying entity as a FK.

---

### 1.1 USER

| Attribute     | Data Type    | Constraint       |
|---------------|--------------|------------------|
| user_id       | SERIAL       | PRIMARY KEY      |
| name          | VARCHAR(100) | NOT NULL         |
| email         | VARCHAR(150) | NOT NULL, UNIQUE |
| password_hash | VARCHAR(255) | NOT NULL         |

### 1.2 PASSENGER (subtype of USER)

| Attribute | Data Type   | Constraint                      |
|-----------|-------------|---------------------------------|
| user_id   | INTEGER     | PRIMARY KEY, FK → USER(user_id) |
| phone     | VARCHAR(15) | NOT NULL                        |
| dob       | DATE        | NOT NULL                        |
| gender    | CHAR(1)     | CHECK (M/F/O)                   |

### 1.3 ADMIN (subtype of USER)

| Attribute   | Data Type   | Constraint                      |
|-------------|-------------|---------------------------------|
| user_id     | INTEGER     | PRIMARY KEY, FK → USER(user_id) |
| employee_id | VARCHAR(20) | NOT NULL, UNIQUE                |
| role        | VARCHAR(50) | NOT NULL                        |

### 1.4 STATION

| Attribute    | Data Type    | Constraint       |
|--------------|--------------|------------------|
| station_id   | SERIAL       | PRIMARY KEY      |
| station_code | CHAR(5)      | NOT NULL, UNIQUE |
| station_name | VARCHAR(100) | NOT NULL         |

|      |             |          |
|------|-------------|----------|
| city | VARCHAR(80) | NOT NULL |
|------|-------------|----------|

## 1.5 TRAIN

| Attribute         | Data Type    | Constraint                         |
|-------------------|--------------|------------------------------------|
| train_number      | INTEGER      | PRIMARY KEY                        |
| train_name        | VARCHAR(100) | NOT NULL                           |
| source_station_id | INTEGER      | NOT NULL, FK → STATION(station_id) |
| dest_station_id   | INTEGER      | NOT NULL, FK → STATION(station_id) |

## 1.6 TRAIN\_CLASS

| Attribute  | Data Type     | Constraint       |
|------------|---------------|------------------|
| class_id   | SERIAL        | PRIMARY KEY      |
| class_code | VARCHAR(5)    | NOT NULL, UNIQUE |
| class_name | VARCHAR(50)   | NOT NULL         |
| base_price | NUMERIC(10,2) | NOT NULL         |

## 1.7 ROUTE\_STOP

| Attribute      | Data Type | Constraint                         |
|----------------|-----------|------------------------------------|
| route_stop_id  | SERIAL    | PRIMARY KEY                        |
| train_number   | INTEGER   | NOT NULL, FK → TRAIN(train_number) |
| station_id     | INTEGER   | NOT NULL, FK → STATION(station_id) |
| stop_sequence  | INTEGER   | NOT NULL                           |
| arrival_time   | TIME      | NULL (source station)              |
| departure_time | TIME      | NULL (destination station)         |

Candidate Key: (train\_number, stop\_sequence) and (train\_number, station\_id)

## 1.8 COACH

| Attribute | Data Type | Constraint  |
|-----------|-----------|-------------|
| coach_id  | SERIAL    | PRIMARY KEY |

|              |            |                                      |
|--------------|------------|--------------------------------------|
| train_number | INTEGER    | NOT NULL, FK → TRAIN(train_number)   |
| class_id     | INTEGER    | NOT NULL, FK → TRAIN_CLASS(class_id) |
| coach_number | VARCHAR(5) | NOT NULL                             |
| total_seats  | INTEGER    | NOT NULL                             |

## 1.9 SEAT

| Attribute   | Data Type   | Constraint                     |
|-------------|-------------|--------------------------------|
| seat_id     | SERIAL      | PRIMARY KEY                    |
| coach_id    | INTEGER     | NOT NULL, FK → COACH(coach_id) |
| seat_number | VARCHAR(5)  | NOT NULL                       |
| berth_type  | VARCHAR(15) | NOT NULL                       |

## 1.10 BOOKING

| Attribute         | Data Type     | Constraint                           |
|-------------------|---------------|--------------------------------------|
| booking_id        | SERIAL        | PRIMARY KEY                          |
| passenger_id      | INTEGER       | NOT NULL, FK → PASSENGER(user_id)    |
| train_number      | INTEGER       | NOT NULL, FK → TRAIN(train_number)   |
| class_id          | INTEGER       | NOT NULL, FK → TRAIN_CLASS(class_id) |
| source_station_id | INTEGER       | NOT NULL, FK → STATION(station_id)   |
| dest_station_id   | INTEGER       | NOT NULL, FK → STATION(station_id)   |
| travel_date       | DATE          | NOT NULL                             |
| pnr               | VARCHAR(15)   | NOT NULL, UNIQUE                     |
| status            | VARCHAR(15)   | NOT NULL                             |
| total_fare        | NUMERIC(10,2) | NOT NULL                             |
| booked_at         | TIMESTAMP     | DEFAULT NOW()                        |

## 1.11 TICKET\_PASSENGER (weak entity)

Represents each individual traveller on a booking. One booking (PNR) can have multiple passengers, each with their own status (Confirmed / WL / Cancelled) and seat assignment.

| Attribute      | Data Type    | Constraint                         |
|----------------|--------------|------------------------------------|
| ticket_id      | SERIAL       | PRIMARY KEY                        |
| booking_id     | INTEGER      | NOT NULL, FK → BOOKING(booking_id) |
| seat_id        | INTEGER      | NULL, FK → SEAT(seat_id)           |
| passenger_name | VARCHAR(100) | NOT NULL                           |
| age            | INTEGER      | NOT NULL                           |
| gender         | CHAR(1)      | NOT NULL                           |
| ticket_status  | VARCHAR(12)  | NOT NULL (CNF/WL/CAN)              |
| wl_number      | INTEGER      | NULL (set when WL)                 |

## 1.12 PAYMENT

| Attribute          | Data Type     | Constraint                                 |
|--------------------|---------------|--------------------------------------------|
| payment_id         | SERIAL        | PRIMARY KEY                                |
| booking_id         | INTEGER       | NOT NULL, UNIQUE, FK → BOOKING(booking_id) |
| payment_service    | VARCHAR(50)   | NOT NULL                                   |
| acknowledgment_ref | VARCHAR(100)  | NOT NULL                                   |
| payment_method     | VARCHAR(30)   | NOT NULL                                   |
| amount             | NUMERIC(10,2) | NOT NULL                                   |
| payment_status     | VARCHAR(15)   | NOT NULL                                   |
| paid_at            | TIMESTAMP     | DEFAULT NOW()                              |

## 1.13 CANCELLATION

| Attribute       | Data Type     | Constraint                                 |
|-----------------|---------------|--------------------------------------------|
| cancellation_id | SERIAL        | PRIMARY KEY                                |
| booking_id      | INTEGER       | NOT NULL, UNIQUE, FK → BOOKING(booking_id) |
| cancelled_at    | TIMESTAMP     | DEFAULT NOW()                              |
| refund_amount   | NUMERIC(10,2) | NOT NULL                                   |
| refund_status   | VARCHAR(15)   | NOT NULL                                   |

## 1.14 FEEDBACK

| Attribute    | Data Type | Constraint                         |
|--------------|-----------|------------------------------------|
| feedback_id  | SERIAL    | PRIMARY KEY                        |
| passenger_id | INTEGER   | NOT NULL, FK → PASSENGER(user_id)  |
| booking_id   | INTEGER   | NOT NULL, FK → BOOKING(booking_id) |
| rating       | INTEGER   | CHECK (1-5)                        |
| review       | TEXT      | NULL                               |
| submitted_at | TIMESTAMP | DEFAULT NOW()                      |

## Part 2: Minimal Functional Dependency Set

A minimal (canonical) FD set has: (a) single attribute on right-hand side, (b) no extraneous attributes on left-hand side, and (c) no redundant FDs. Below is the complete minimal FD set for all 14 relations.

---

### USER

```
user_id → name
user_id → email
user_id → password_hash
email → user_id    [alternate key]
```

### PASSENGER

```
user_id → phone
user_id → dob
user_id → gender
```

### ADMIN

```
user_id → employee_id
user_id → role
employee_id → user_id    [alternate key]
```

### STATION

```
station_id → station_code
station_id → station_name
station_id → city
station_code → station_id    [alternate key]
```

### TRAIN

```
train_number → train_name
train_number → source_station_id
train_number → dest_station_id
```

### TRAIN\_CLASS

```
class_id → class_code
class_id → class_name
class_id → base_price
```

class\_code → class\_id [alternate key]

## ROUTE\_STOP

{train\_number, stop\_sequence} → station\_id  
{train\_number, stop\_sequence} → arrival\_time  
{train\_number, stop\_sequence} → departure\_time  
{train\_number, station\_id} → stop\_sequence [alt key]

## COACH

coach\_id → train\_number  
coach\_id → class\_id  
coach\_id → coach\_number  
coach\_id → total\_seats  
{train\_number, coach\_number} → coach\_id [alt key]

## SEAT

seat\_id → coach\_id  
seat\_id → seat\_number  
seat\_id → berth\_type  
{coach\_id, seat\_number} → seat\_id [alt key]

## BOOKING

booking\_id → passenger\_id  
booking\_id → train\_number  
booking\_id → class\_id  
booking\_id → source\_station\_id  
booking\_id → dest\_station\_id  
booking\_id → travel\_date  
booking\_id → pnr  
booking\_id → status  
booking\_id → total\_fare  
booking\_id → booked\_at  
pnr → booking\_id [alternate key]

## TICKET\_PASSENGER

ticket\_id → booking\_id  
ticket\_id → seat\_id  
ticket\_id → passenger\_name  
ticket\_id → age



ticket\_id → gender  
ticket\_id → ticket\_status  
ticket\_id → wl\_number

## PAYMENT

payment\_id → booking\_id  
payment\_id → payment\_service  
payment\_id → acknowledgment\_ref  
payment\_id → payment\_method  
payment\_id → amount  
payment\_id → payment\_status  
payment\_id → paid\_at  
booking\_id → payment\_id [alternate key, 1-to-1]

## CANCELLATION

cancellation\_id → booking\_id  
cancellation\_id → cancelled\_at  
cancellation\_id → refund\_amount  
cancellation\_id → refund\_status  
booking\_id → cancellation\_id [alternate key]

## FEEDBACK

feedback\_id → passenger\_id  
feedback\_id → booking\_id  
feedback\_id → rating  
feedback\_id → review  
feedback\_id → submitted\_at  
booking\_id → feedback\_id [alternate key, 0..1]

## Part 3: Normalization Proofs (BCNF)

As per the professor's definition: A relation R is in BCNF when, for every FD  $X \rightarrow Y$  that holds on R, X is a superkey of R. We apply this check to every relation in our schema.

---

### 3.1 USER

|                                |                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------|
| <b>Relation</b>                | USER(user_id, name, email, password_hash)                                                    |
| <b>Key(s)</b>                  | user_id [also email is alternate key]                                                        |
| <b>Functional Dependencies</b> | user_id → name, user_id → email, user_id → password_hash, email → user_id                    |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                |
| <b>Reason</b>                  | Both user_id and email are keys. Every FD has a key as its determinant. No FD violates BCNF. |

### 3.2 PASSENGER

|                                |                                                                               |
|--------------------------------|-------------------------------------------------------------------------------|
| <b>Relation</b>                | PASSENGER(user_id, phone, dob, gender)                                        |
| <b>Key(s)</b>                  | user_id                                                                       |
| <b>Functional Dependencies</b> | user_id → phone, user_id → dob, user_id → gender                              |
| <b>In BCNF?</b>                | YES — In BCNF                                                                 |
| <b>Reason</b>                  | user_id is the only key. Every FD has user_id as determinant. BCNF satisfied. |

### 3.3 ADMIN

|                                |                                                                                      |
|--------------------------------|--------------------------------------------------------------------------------------|
| <b>Relation</b>                | ADMIN(user_id, employee_id, role)                                                    |
| <b>Key(s)</b>                  | user_id [also employee_id is alternate key]                                          |
| <b>Functional Dependencies</b> | user_id → employee_id, user_id → role, employee_id → user_id                         |
| <b>In BCNF?</b>                | YES — In BCNF                                                                        |
| <b>Reason</b>                  | Both user_id and employee_id are keys. All FD determinants are keys. BCNF satisfied. |

### 3.4 STATION

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <b>Relation</b> | STATION(station_id, station_code, station_name, city) |
|-----------------|-------------------------------------------------------|

|                                |                                                                                            |
|--------------------------------|--------------------------------------------------------------------------------------------|
| <b>Key(s)</b>                  | station_id [also station_code is alternate key]                                            |
| <b>Functional Dependencies</b> | station_id→station_code, station_id→station_name, station_id→city, station_code→station_id |
| <b>In BCNF?</b>                | YES — In BCNF                                                                              |
| <b>Reason</b>                  | Both station_id and station_code are keys. Every determinant is a key. BCNF satisfied.     |

### 3.5 TRAIN

|                                |                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------|
| <b>Relation</b>                | TRAIN(train_number, train_name, source_station_id, dest_station_id)                     |
| <b>Key(s)</b>                  | train_number                                                                            |
| <b>Functional Dependencies</b> | train_number→train_name, train_number→source_station_id, train_number→dest_station_id   |
| <b>In BCNF?</b>                | YES — In BCNF                                                                           |
| <b>Reason</b>                  | train_number is the sole key. All FDs have train_number as determinant. BCNF satisfied. |

### 3.6 TRAIN\_CLASS

|                                |                                                                                       |
|--------------------------------|---------------------------------------------------------------------------------------|
| <b>Relation</b>                | TRAIN_CLASS(class_id, class_code, class_name, base_price)                             |
| <b>Key(s)</b>                  | class_id [also class_code is alternate key]                                           |
| <b>Functional Dependencies</b> | class_id→class_code, class_id→class_name, class_id→base_price, class_code→class_id    |
| <b>In BCNF?</b>                | YES — In BCNF                                                                         |
| <b>Reason</b>                  | Both class_id and class_code are keys. Every FD determinant is a key. BCNF satisfied. |

### 3.7 ROUTE\_STOP

|                                |                                                                                                                                       |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Relation</b>                | ROUTE_STOP(route_stop_id, train_number, station_id, stop_sequence, arrival_time, departure_time)                                      |
| <b>Key(s)</b>                  | {train_number, stop_sequence} [also {train_number, station_id}; route_stop_id is surrogate key]                                       |
| <b>Functional Dependencies</b> | {train_number, stop_sequence}→station_id, →arrival_time, →departure_time; {train_number, station_id}→stop_sequence                    |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                                                         |
| <b>Reason</b>                  | All three candidate keys (surrogate route_stop_id, composite {train_number, stop_sequence}, composite {train_number, station_id}) are |

|  |                                                                             |
|--|-----------------------------------------------------------------------------|
|  | determinants of their respective FDs. No non-key FD exists. BCNF satisfied. |
|--|-----------------------------------------------------------------------------|

### 3.8 COACH

|                                |                                                                                                                             |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Relation</b>                | COACH(coach_id, train_number, class_id, coach_number, total_seats)                                                          |
| <b>Key(s)</b>                  | coach_id [also {train_number, coach_number} is alternate key]                                                               |
| <b>Functional Dependencies</b> | coach_id→train_number, coach_id→class_id, coach_id→coach_number, coach_id→total_seats, {train_number,coach_number}→coach_id |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                                               |
| <b>Reason</b>                  | coach_id and {train_number, coach_number} are both keys. Every FD has a key as determinant. BCNF satisfied.                 |

### 3.9 SEAT

|                                |                                                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Relation</b>                | SEAT(seat_id, coach_id, seat_number, berth_type)                                                 |
| <b>Key(s)</b>                  | seat_id [also {coach_id, seat_number} is alternate key]                                          |
| <b>Functional Dependencies</b> | seat_id→coach_id, seat_id→seat_number, seat_id→berth_type, {coach_id,seat_number}→seat_id        |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                    |
| <b>Reason</b>                  | Both seat_id and {coach_id, seat_number} are keys. All FD determinants are keys. BCNF satisfied. |

### 3.10 BOOKING

|                                |                                                                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Relation</b>                | BOOKING(booking_id, passenger_id, train_number, class_id, source_station_id, dest_station_id, travel_date, pnr, status, total_fare, booked_at) |
| <b>Key(s)</b>                  | booking_id [also pnr is alternate key]                                                                                                         |
| <b>Functional Dependencies</b> | booking_id→{all other attributes}, pnr→booking_id                                                                                              |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                                                                  |
| <b>Reason</b>                  | booking_id and pnr are both keys. Every non-trivial FD has a key as determinant. No partial or transitive dependency exists. BCNF satisfied.   |

### 3.11 TICKET\_PASSENGER

|                 |                                                                                                         |
|-----------------|---------------------------------------------------------------------------------------------------------|
| <b>Relation</b> | TICKET_PASSENGER(ticket_id, booking_id, seat_id, passenger_name, age, gender, ticket_status, wl_number) |
|-----------------|---------------------------------------------------------------------------------------------------------|

|                                |                                                                                                                        |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Key(s)</b>                  | ticket_id                                                                                                              |
| <b>Functional Dependencies</b> | ticket_id → {all other attributes}                                                                                     |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                                          |
| <b>Reason</b>                  | ticket_id is the only key. Every attribute depends only on ticket_id. No non-trivial FD violates BCNF. BCNF satisfied. |

### 3.12 PAYMENT

|                                |                                                                                                                       |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Relation</b>                | PAYMENT(payment_id, booking_id, payment_service, acknowledgment_ref, payment_method, amount, payment_status, paid_at) |
| <b>Key(s)</b>                  | payment_id [also booking_id is alternate key — 1-to-1 with BOOKING]                                                   |
| <b>Functional Dependencies</b> | payment_id → {all}, booking_id → payment_id                                                                           |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                                         |
| <b>Reason</b>                  | Both payment_id and booking_id are keys. All FD determinants are keys. BCNF satisfied.                                |

### 3.13 CANCELLATION

|                                |                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------|
| <b>Relation</b>                | CANCELLATION(cancellation_id, booking_id, cancelled_at, refund_amount, refund_status)        |
| <b>Key(s)</b>                  | cancellation_id [also booking_id is alternate key]                                           |
| <b>Functional Dependencies</b> | cancellation_id → {all}, booking_id → cancellation_id                                        |
| <b>In BCNF?</b>                | YES — In BCNF                                                                                |
| <b>Reason</b>                  | Both cancellation_id and booking_id are keys. Every FD determinant is a key. BCNF satisfied. |

### 3.14 FEEDBACK

|                                |                                                                               |
|--------------------------------|-------------------------------------------------------------------------------|
| <b>Relation</b>                | FEEDBACK(feedback_id, passenger_id, booking_id, rating, review, submitted_at) |
| <b>Key(s)</b>                  | feedback_id [booking_id is alternate key — at most one feedback per booking]  |
| <b>Functional Dependencies</b> | feedback_id → {all}, booking_id → feedback_id                                 |
| <b>In BCNF?</b>                | YES — In BCNF                                                                 |

**Reason**

Both feedback\_id and booking\_id are keys. All FD determinants are keys.  
BCNF satisfied.

**Conclusion: All 14 relations are in BCNF. No decomposition is required. The schema was designed bottom-up from the ER diagram, ensuring each entity and relationship was kept in a single clean relation with no transitive or partial dependencies.**

## Part 4: DDL Scripts — PostgreSQL

The following script creates a dedicated schema named **tts** (Train Transport System) on the PostgreSQL server. Run this script as a single file.

---

### Schema Creation

```
-- =====
-- Railway Transport & Reservation System
-- DDL Script — PostgreSQL
-- Schema: tts
-- Group: 202401068
-- =====

DROP SCHEMA IF EXISTS tts CASCADE;
CREATE SCHEMA tts;
SET search_path TO tts;
```

### Table: USER

```
CREATE TABLE tts."user" (
    user_id      SERIAL          PRIMARY KEY,
    name          VARCHAR(100)    NOT NULL,
    email         VARCHAR(150)    NOT NULL UNIQUE,
    password_hash VARCHAR(255)    NOT NULL
);
```

### Table: PASSENGER

```
CREATE TABLE tts.passenger (
    user_id  INTEGER          PRIMARY KEY
            REFERENCES tts."user"(user_id)
            ON DELETE CASCADE,

    phone    VARCHAR(15) NOT NULL,
    dob       DATE         NOT NULL,
    gender    CHAR(1)      CHECK (gender IN ('M', 'F', 'O'))
);
```

### Table: ADMIN

```
CREATE TABLE tts.admin (
    user_id  INTEGER          PRIMARY KEY
            REFERENCES tts."user"(user_id)
            ON DELETE CASCADE,
```

```
        employee_id VARCHAR(20) NOT NULL UNIQUE,  
        role          VARCHAR(50) NOT NULL  
    );
```

## Table: STATION

```
CREATE TABLE tts.station (  
    station_id  SERIAL          PRIMARY KEY,  
    station_code CHAR(5)        NOT NULL UNIQUE,  
    station_name VARCHAR(100) NOT NULL,  
    city        VARCHAR(80) NOT NULL  
);
```

## Table: TRAIN

```
CREATE TABLE tts.train (  
    train_number      INTEGER          PRIMARY KEY,  
    train_name        VARCHAR(100) NOT NULL,  
    source_station_id INTEGER          NOT NULL  
                                REFERENCES tts.station(station_id),  
    dest_station_id   INTEGER          NOT NULL  
                                REFERENCES tts.station(station_id)  
);
```

## Table: TRAIN\_CLASS

```
CREATE TABLE tts.train_class (  
    class_id  SERIAL          PRIMARY KEY,  
    class_code VARCHAR(5)      NOT NULL UNIQUE,  
    class_name VARCHAR(50)     NOT NULL,  
    base_price NUMERIC(10,2)   NOT NULL  
);
```

## Table: ROUTE\_STOP

```
CREATE TABLE tts.route_stop (  
    route_stop_id SERIAL          PRIMARY KEY,  
    train_number   INTEGER          NOT NULL  
                                REFERENCES tts.train(train_number),  
    station_id     INTEGER          NOT NULL  
                                REFERENCES tts.station(station_id),  
    stop_sequence  INTEGER          NOT NULL,  
    arrival_time   TIME,  
    departure_time TIME,  
    UNIQUE (train_number, stop_sequence),
```



```
        UNIQUE (train_number, station_id)
    );
```

## Table: COACH

```
CREATE TABLE tts.coach (
    coach_id      SERIAL          PRIMARY KEY,
    train_number  INTEGER          NOT NULL
                        REFERENCES tts.train(train_number),
    class_id      INTEGER          NOT NULL
                        REFERENCES tts.train_class(class_id),
    coach_number  VARCHAR(5)       NOT NULL,
    total_seats   INTEGER          NOT NULL,
    UNIQUE (train_number, coach_number)
);
```

## Table: SEAT

```
CREATE TABLE tts.seat (
    seat_id       SERIAL          PRIMARY KEY,
    coach_id      INTEGER          NOT NULL
                        REFERENCES tts.coach(coach_id),
    seat_number   VARCHAR(5)       NOT NULL,
    berth_type    VARCHAR(15)      NOT NULL,
    UNIQUE (coach_id, seat_number)
);
```

## Table: BOOKING

```
CREATE TABLE tts.booking (
    booking_id     SERIAL          PRIMARY KEY,
    passenger_id   INTEGER          NOT NULL
                        REFERENCES tts.passenger(user_id),
    train_number   INTEGER          NOT NULL
                        REFERENCES tts.train(train_number),
    class_id       INTEGER          NOT NULL
                        REFERENCES tts.train_class(class_id),
    source_station_id INTEGER          NOT NULL
                        REFERENCES tts.station(station_id),
    dest_station_id INTEGER          NOT NULL
                        REFERENCES tts.station(station_id),
    travel_date    DATE             NOT NULL,
    pnr            VARCHAR(15)       NOT NULL UNIQUE,
    status         VARCHAR(15)       NOT NULL
                        CHECK (status IN ('CONFIRMED', 'WAITING', 'CANCELLED')),
    total_fare     NUMERIC(10,2)     NOT NULL,
```

```

        booked_at          TIMESTAMP          DEFAULT NOW()
    );

```

## Table: TICKET\_PASSENGER (weak entity)

```

CREATE TABLE tts.ticket_passenger (
    ticket_id      SERIAL          PRIMARY KEY,
    booking_id     INTEGER         NOT NULL
                        REFERENCES tts.booking(booking_id),
    seat_id        INTEGER
                        REFERENCES tts.seat(seat_id),
    passenger_name VARCHAR(100) NOT NULL,
    age            INTEGER         NOT NULL,
    gender         CHAR(1)        NOT NULL
                        CHECK (gender IN ('M','F','O')),
    ticket_status  VARCHAR(12) NOT NULL
                        CHECK (ticket_status IN ('CONFIRMED','WL','CANCELLED')),
    wl_number      INTEGER
);

```

## Table: PAYMENT

```

CREATE TABLE tts.payment (
    payment_id      SERIAL          PRIMARY KEY,
    booking_id      INTEGER         NOT NULL UNIQUE
                        REFERENCES tts.booking(booking_id),
    payment_service VARCHAR(50)     NOT NULL,
    acknowledgment_ref VARCHAR(100) NOT NULL,
    payment_method  VARCHAR(30)     NOT NULL,
    amount          NUMERIC(10,2)   NOT NULL,
    payment_status  VARCHAR(15)     NOT NULL
                        CHECK (payment_status IN
('SUCCESS','PENDING','FAILED')),
    paid_at         TIMESTAMP        DEFAULT NOW()
);

```

## Table: CANCELLATION

```

CREATE TABLE tts.cancellation (
    cancellation_id SERIAL          PRIMARY KEY,
    booking_id      INTEGER         NOT NULL UNIQUE
                        REFERENCES tts.booking(booking_id),
    cancelled_at    TIMESTAMP        DEFAULT NOW(),
    refund_amount   NUMERIC(10,2)   NOT NULL,
    refund_status   VARCHAR(15)     NOT NULL
                        CHECK (refund_status IN ('PENDING','PROCESSED','FAILED'))
);

```

```
);
```

## Table: FEEDBACK

```
CREATE TABLE tts.feedback (  
    feedback_id SERIAL PRIMARY KEY,  
    passenger_id INTEGER NOT NULL  
        REFERENCES tts.passenger(user_id),  
    booking_id INTEGER NOT NULL UNIQUE  
        REFERENCES tts.booking(booking_id),  
    rating INTEGER CHECK (rating BETWEEN 1 AND 5),  
    review TEXT,  
    submitted_at TIMESTAMP DEFAULT NOW()  
);
```

## Indexes for Performance

```
-- Speed up train search by route  
CREATE INDEX idx_route_stop_train  
    ON tts.route_stop(train_number, stop_sequence);  
  
-- Speed up PNR lookups  
CREATE INDEX idx_booking_pnr  
    ON tts.booking(pnr);  
  
-- Speed up passenger booking history  
CREATE INDEX idx_booking_passenger  
    ON tts.booking(passenger_id);  
  
-- Speed up waiting list queries per train/class/date  
CREATE INDEX idx_ticket_status  
    ON tts.ticket_passenger(booking_id, ticket_status);  
  
-- Speed up seat availability check  
CREATE INDEX idx_seat_coach  
    ON tts.seat(coach_id);
```

## End of DDL Script

```
-- All tables created in schema tts  
-- Total tables: 14  
-- All relations proven to be in BCNF  
-- =====
```